LAPORAN TUGAS 1

GENERIC ADT DAN HASH TABLE

Disusun oleh:

Nama: Hanura Nur Ridwan
NIM: 253040086
Program Studi: Teknik Informatika
Fakultas: Teknik
Universitas: Universitas Pasundan

BAB I – PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu bagian penting dalam pengembangan program karena digunakan untuk mengatur dan mengelola informasi. Dalam tugas ini, konsep Generic Abstract Data Type (ADT) dan Hash Table diterapkan untuk membuat sistem sederhana pengelolaan data tiket layanan.

Generic ADT digunakan untuk menyimpan riwayat data tiket, sedangkan hash table digunakan untuk menyimpan indeks tiket berdasarkan ID. Dua metode hash table yang diterapkan adalah Separate Chaining dan Linear Probing.

1.2 Tujuan

Tujuan pengerjaan tugas ini adalah:

1. Memahami penerapan Generic ADT menggunakan Java.
2. Mengimplementasikan `SimpleBag<E>` dan `ArrayBag<E>`.
3. Memahami dan menerapkan metode Separate Chaining.
4. Memahami dan menerapkan metode Linear Probing.
5. Mengintegrasikan struktur data untuk mengelola riwayat dan indeks tiket.
6. Melakukan pengujian terhadap operasi dan perilaku struktur data.

BAB II – IMPLEMENTASI PROGRAM

2.1 Generic ADT

Generic ADT diterapkan melalui interface `SimpleBag<E>` dan kelas `ArrayBag<E>`. Struktur ini digunakan untuk menyimpan kumpulan data dengan tipe yang dapat ditentukan.

Implementasi menyediakan operasi `add`, `contains`, `size`, dan `iterator`. Data disimpan sesuai urutan penambahan, serta memungkinkan elemen duplikat.

Penggunaan generic ditunjukkan melalui penyimpanan data bertipe `String` dan objek `Tiket`.

2.2 MiniMap

`MiniMap<K,V>` digunakan sebagai interface yang mendefinisikan operasi dasar hash map, yaitu `put`, `get`, `remove`, `containsKey`, dan `size`.

Interface ini menjadi dasar bagi dua implementasi hash table yang digunakan dalam program.

2.3 Separate Chaining

Separate Chaining diimplementasikan melalui kelas `ChainedHashMap`. Metode ini menggunakan bucket untuk menyimpan node yang memiliki indeks hash sama.

Ketika terjadi collision, data baru ditempatkan pada rantai node di bucket yang sama. Dengan demikian, beberapa key dapat disimpan meskipun memiliki indeks bucket yang sama.

2.4 Linear Probing

Linear Probing diimplementasikan melalui kelas `ProbeHashMap`. Metode ini menangani collision dengan memeriksa slot berikutnya sampai menemukan posisi yang dapat digunakan atau menemukan key yang dicari.

Implementasi menggunakan tiga status slot, yaitu `EMPTY`, `OCCUPIED`, dan `DELETED`. Status `DELETED` digunakan sebagai tombstone agar proses pencarian tetap dapat berjalan dengan benar setelah penghapusan data.

2.5 Integrasi Data Tiket

Kelas `Tiket` digunakan untuk merepresentasikan data tiket yang memiliki ID, nama, dan status.

Data tiket dibuat immutable sehingga perubahan informasi menghasilkan objek baru tanpa mengubah versi sebelumnya.

Program mengintegrasikan `ArrayBag` sebagai penyimpan riwayat dan hash map sebagai indeks data tiket terkini. Riwayat tetap menyimpan versi lama meskipun data pada indeks diperbarui atau dihapus.

BAB III – PENGUJIAN DAN HASIL

3.1 Pengujian Generic ADT

Pengujian Generic ADT dilakukan untuk memeriksa operasi penambahan, pencarian, ukuran, dan iterator. Pengujian juga mencakup elemen duplikat, nilai null, dua iterator independen, iterator yang telah habis, serta perubahan data setelah iterator dibuat.

Hasil pengujian: [Isi berdasarkan hasil yang benar-benar muncul di terminal.]

3.2 Pengujian Chained Hash Map

Pengujian dilakukan terhadap operasi penambahan, pencarian, pembaruan, penghapusan, collision, dan perubahan kapasitas.

Hasil pengujian: [Isi berdasarkan hasil yang benar-benar muncul di terminal.]

3.3 Pengujian Probe Hash Map

Pengujian dilakukan terhadap operasi penambahan, pencarian, pembaruan, penghapusan, collision, penggunaan tombstone, dan kondisi kapasitas tabel.

Hasil pengujian: [Isi berdasarkan hasil yang benar-benar muncul di terminal.]

3.4 Pengujian Jejak Linear Probing

Pengujian khusus dilakukan menggunakan kapasitas 5 dengan rehash otomatis dinonaktifkan.

Urutan operasi:

1. `put(2, 2)`

2. `put(7, 7)`
3. `put(12, 12)`
4. `remove(7)`
5. `get(12)`
6. `put(12, 99)`
7. `put(17, 17)`

Hasil akhir yang diharapkan adalah:

- Size akhir: 3.
- Key 12 memiliki value 99.
- Key 7 sudah tidak tersedia.
- Penyisipan key 17 menggunakan kembali slot tombstone.

Hasil aktual: [Isi berdasarkan hasil pengujian program.]

3.5 Pengujian Integrasi Tiket

Pengujian integrasi dilakukan pada kedua implementasi hash map menggunakan data tiket yang sama.

Urutan proses meliputi penambahan tiket dengan ID 2, 7, dan 12, penghapusan tiket ID 7, pencarian tiket ID 12, pembaruan tiket ID 12, serta penambahan tiket ID 17.

Hasil akhir yang diharapkan:

ID	Nama	Status
2	A	BARU
12	C revisi	DIPROSES
17	D	BARU

Indeks memiliki 3 data aktif. Riwayat tetap menyimpan 5 versi hasil operasi `put`, termasuk versi lama tiket ID 12.

Hasil aktual: [Isi berdasarkan hasil pengujian program.]

3.6 Bukti Pengujian

```
--- CHAINED HASH MAP ---

1. Menambahkan Tiket
Tiket berhasil ditambahkan.

2. Menghapus Tiket ID 7
Hasil remove: Tiket{id=7, nama='Budi', status='BARU'}
Tiket ID 7 masih di map: false

3. Mengambil Tiket ID 12
Tiket ditemukan: Tiket{id=12, nama='Citra', status='BARU'}

4. Memperbarui Tiket ID 12
Versi lama: Tiket{id=12, nama='Citra', status='BARU'}
Versi baru: Tiket{id=12, nama='Citra revisi', status='DIPROSES'}

5. Menambahkan Tiket ID 17
Tiket berhasil ditambahkan.
```

```
--- PROBE HASH MAP ---

1. Menambahkan Tiket
Tiket berhasil ditambahkan.

2. Menghapus Tiket ID 7
Hasil remove: Tiket{id=7, nama='Budi', status='BARU'}
Tiket ID 7 masih di map: false

3. Mengambil Tiket ID 12
Tiket ditemukan: Tiket{id=12, nama='Citra', status='BARU'}

4. Memperbarui Tiket ID 12
Versi lama: Tiket{id=12, nama='Citra', status='BARU'}
Versi baru: Tiket{id=12, nama='Citra revisi', status='DIPROSES'}

5. Menambahkan Tiket ID 17
Tiket berhasil ditambahkan.
```

```
    PENGUJIAN PROBE HASH MAP

1. Memasukkan Data
Put 2: null
Put 7: null
Put 12: null
Size: 3

2. Menghapus Key 7
Remove 7: 7
Contains 7: false

3. Mengakses Key 12
Get 12: 12

4. Memperbarui Key 12
Old Value: 12
New Value: 99

5. Memasukkan Key 17
Put 17: null

6. Hasil Akhir
Size: 3
Key 2: 2
Key 12: 99
Key 17: 17
Contains 7: false

=== PENGUJIAN PROBE SELESAI ===
```

```
    PENGUJIAN CHAINED HASH MAP

1. Pengujian Put
Put 2: null
Put 7: null
Put 12: null
Size: 3

2. Pengujian Get
Get 2: Andi
Get 7: Budi
Get 12: Citra

3. Pengujian Update
Old Value: Budi
New Value: Budi Revisi
Size: 3

4. Pengujian ContainsKey
Key 7: true
Key 99: false

5. Pengujian Remove
Remove 2: Andi
Contains 2: false
Final Size: 2

6. Pengujian Resize
Size setelah penambahan: 4
Get 12: Citra
Get 17: Dina
Get 22: Eko

=== PENGUJIAN CHAINED SELESAI ===
```

Bukti pengujian berupa tangkapan layar hasil eksekusi program disertakan pada folder `docs/bukti-pengujian/`.

BAB IV – KESIMPULAN

Berdasarkan pengerjaan tugas ini, Generic ADT dapat digunakan untuk menyimpan kumpulan data dengan tipe yang fleksibel. Implementasi `ArrayBag` juga memungkinkan penyimpanan riwayat perubahan data tiket.

Selain itu, Separate Chaining dan Linear Probing merupakan dua metode yang dapat digunakan untuk mengelola indeks berdasarkan key. Keduanya memiliki cara penanganan collision yang berbeda, tetapi sama-sama dapat digunakan untuk menjalankan operasi penambahan, pencarian, pembaruan, dan penghapusan data.

Melalui integrasi kedua struktur data tersebut, program dapat mempertahankan riwayat versi tiket sekaligus menyediakan indeks yang berisi data tiket terkini.
